

STAT 408 Lab 2: dplyr

Hadley Stabnau

2026-01-22

Section 3.2.5

1. In a single pipeline for each condition, find all the flights that meet the condition:
 - Had an arrival delay of two or more hours: 10,200 flights
 - Flew to Houston (IAH or HOU): 9,313 flights
 - Were operated by United, American or Delta: 139,504 flights
 - Departed in summer (July, August, and September): 86,995 flights
 - Arrived more than two hours late but didn't leave late: 210,094 flights
 - Were delayed by at least an hour, but made up over 30 minutes in flight: 45,380 flights

```
#install.packages("nycflights13")
library(nycflights13)
library(tidyverse)
glimpse(flights)
```

Rows: 336,776

Columns: 19

```
$ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ~
$ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ~
$ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -1~
$ arr_time  <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849,~
$ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851,~
$ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -1~
$ carrier   <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "~
$ flight    <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4~
$ tailnum   <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394~
```

```

$ origin      <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA", ~
$ dest        <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD", ~
$ air_time    <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1~
$ distance    <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ~
$ hour        <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6~
$ minute      <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0~
$ time_hour   <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-01 0~

```

```

flights |>
  count(arr_delay>=120)

```

```

# A tibble: 3 x 2
  `arr_delay >= 120`      n
  <lgl>              <int>
1 FALSE             317146
2 TRUE              10200
3 NA                 9430

```

```

flights |>
  filter(dest=="IAH" |dest=="HOU") |>
  count(dest)

```

```

# A tibble: 2 x 2
  dest      n
  <chr> <int>
1 HOU   2115
2 IAH   7198

```

```

flights |>
  filter(carrier=="UA" |carrier=="AA" |carrier=="DL") |>
  count(carrier)

```

```

# A tibble: 3 x 2
  carrier      n
  <chr>   <int>
1 AA     32729
2 DL     48110
3 UA     58665

```

```
flights |>
  filter(month==6 | month==7 | month==8) |>
  count(month)
```

```
# A tibble: 3 x 2
  month     n
  <int> <int>
1     6 28243
2     7 29425
3     8 29327
```

```
flights |>
  count(arr_delay>120 | dep_delay<=0)
```

```
# A tibble: 3 x 2
  `arr_delay > 120 | dep_delay <= 0`     n
  <lgl>                                <int>
1 FALSE                                117740
2 TRUE                                 210094
3 NA                                   8942
```

```
flights |>
  count(dep_delay>=60 | (dep_delay-arr_delay)>=30)
```

```
# A tibble: 3 x 2
  `dep_delay >= 60 | (dep_delay - arr_delay) >= 30`     n
  <lgl>                                                <int>
1 FALSE                                                282223
2 TRUE                                                  45380
3 NA                                                    9173
```

- Sort `flights` to find the flights with the longest departure delays. Find the flights that left earliest in the morning.

```
flights |>
  arrange(desc(dep_delay))
```

```
# A tibble: 336,776 x 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
```

```

      <int> <int> <int>      <int>      <int>      <dbl>      <int>      <int>
1  2013      1      9      641      900      1301      1242      1530
2  2013      6     15     1432     1935      1137      1607      2120
3  2013      1     10     1121     1635      1126      1239      1810
4  2013      9     20     1139     1845      1014      1457      2210
5  2013      7     22      845     1600      1005      1044      1815
6  2013      4     10     1100     1900       960      1342      2211
7  2013      3     17     2321      810       911       135      1020
8  2013      6     27      959     1900      899      1236      2226
9  2013      7     22     2257      759      898       121      1026
10 2013     12      5      756     1700      896      1058      2020
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

```

flights |>
  arrange(dep_time)

```

```

# A tibble: 336,776 x 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>      <int>      <dbl>      <int>      <int>
1  2013      1    13         1      2249         72       108       2357
2  2013      1    31         1      2100        181       124       2225
3  2013     11    13         1      2359          2       442       440
4  2013     12    16         1      2359          2       447       437
5  2013     12    20         1      2359          2       430       440
6  2013     12    26         1      2359          2       437       440
7  2013     12    30         1      2359          2       441       437
8  2013      2    11         1      2100        181       111       2225
9  2013      2    24         1      2245         76       121       2354
10 2013      3      8         1      2355          6       431       440
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

- Sort `flights` to find the fastest flights. (Hint: Try including a math calculation inside of your function).

```
flights|>
  arrange(arr_time-dep_time)
```

```
# A tibble: 336,776 x 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     7    17    2400           2142          138     54           2259
2  2013    12     9    2400           2250           70     59           2356
3  2013     6    12    2338           2129          129    17           2235
4  2013    12    29    2332           2155           97    14           2300
5  2013    11     6    2335           2215           80    18           2317
6  2013     2    25    2347           2145          122    30           2239
7  2013     8    13    2351           2152          119    35           2258
8  2013    10    11    2342           2030          192    27           2205
9  2013     2    26    2356           2000          236    41           2104
10 2013     1    24    2342           2159          103    28           2300
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

4. Was there a flight every day of 2013?

```
flights |>
  distinct(year, month, day) |>
  count()
```

```
# A tibble: 1 x 1
   n
   <int>
1  365
```

Yes, according to the above table, there are 365 days out of the year 2013 that a flight occurred.

5. Which flights traveled the farthest distance? Which traveled the least distance?

```
flights |>
  arrange(desc(distance))
```

```
# A tibble: 336,776 x 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     857             900         -3     1516         1530
2  2013     1     2     909             900          9     1525         1530
3  2013     1     3     914             900         14     1504         1530
4  2013     1     4     900             900          0     1516         1530
5  2013     1     5     858             900         -2     1519         1530
6  2013     1     6    1019             900         79     1558         1530
7  2013     1     7    1042             900        102     1620         1530
8  2013     1     8     901             900          1     1504         1530
9  2013     1     9     641             900       1301     1242         1530
10 2013     1    10     859             900         -1     1449         1530
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
flights |>
  arrange(distance)
```

```
# A tibble: 336,776 x 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     7    27      NA             106          NA      NA           245
2  2013     1     3    2127             2129         -2     2222         2224
3  2013     1     4    1240             1200         40     1333         1306
4  2013     1     4    1829             1615        134     1937         1721
5  2013     1     4    2128             2129         -1     2218         2224
6  2013     1     5    1155             1200         -5     1241         1306
7  2013     1     6    2125             2129         -4     2224         2224
8  2013     1     7    2124             2129         -5     2212         2224
9  2013     1     8    2127             2130         -3     2304         2225
10 2013     1     9    2126             2129         -3     2217         2224
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- Does it matter what order you used the `filter()` and `arrange()` if you're using both? Why/Why not? Think about the results. and how much work the functions would have to do.

It does not matter what order you use `filter()` and `arrange()` when using both because either way, the data will be filtered out for only the observations we're interested in and arranged in some order. Whether you filter the data out first or not, shouldn't matter. See the example below where we filter the data so that we are only looking at flights from February and we are looking at the flights with the shortest air times:

```
flights |>
  filter(month==2) |>
  arrange(air_time)
```

```
# A tibble: 24,951 x 19
```

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time
	<int>	<int>	<int>	<int>	<int>	<dbl>	<int>	<int>
1	2013	2	3	2153	2129	24	2247	2224
2	2013	2	5	1303	1315	-12	1342	1411
3	2013	2	12	2123	2130	-7	2211	2225
4	2013	2	3	1251	1257	-6	1337	1354
5	2013	2	7	1313	1315	-2	1359	1411
6	2013	2	11	1313	1315	-2	1422	1411
7	2013	2	16	1308	1313	-5	1347	1414
8	2013	2	23	1307	1313	-6	1346	1414
9	2013	2	24	1315	1320	-5	1353	1419
10	2013	2	25	1314	1316	-2	1352	1413

```
# i 24,941 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
flights |>
  arrange(air_time) |>
  filter(month==2)
```

```
# A tibble: 24,951 x 19
```

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time
	<int>	<int>	<int>	<int>	<int>	<dbl>	<int>	<int>
1	2013	2	3	2153	2129	24	2247	2224
2	2013	2	5	1303	1315	-12	1342	1411
3	2013	2	12	2123	2130	-7	2211	2225
4	2013	2	3	1251	1257	-6	1337	1354
5	2013	2	7	1313	1315	-2	1359	1411
6	2013	2	11	1313	1315	-2	1422	1411

```

7 2013      2    16    1308          1313      -5    1347          1414
8 2013      2    23    1307          1313      -6    1346          1414
9 2013      2    24    1315          1320      -5    1353          1419
10 2013     2    25    1314          1316      -2    1352          1413
# i 24,941 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

Section 3.3.5

1. Compare `dep_time`, `sched_dep_time`, and `dep_delay`. How would you expect those three numbers to be related?

I would expect these three numbers to be related because when `dep_delay` increases, that increase will be equal to the difference between `sched_dep_time` and `dep_time`.

```

flights |>
  arrange(flight)

```

```

# A tibble: 336,776 x 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>      <dbl>   <int>         <int>
1  2013     1     1     856           900        -4    1226           1220
2  2013     1     1    1153          1123         30    1454           1425
3  2013     1     2     855           900        -5    1225           1220
4  2013     1     2    1134          1123         11    1449           1425
5  2013     1     3     855           900        -5    1144           1220
6  2013     1     3    1201          1123         38    1510           1425
7  2013     1     4     858           900        -2    1210           1220
8  2013     1     4    1130          1123         7     1427           1425
9  2013     1     4    2030          2030         0     2313           2338
10 2013     1     5     851           900        -9    1206           1220
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

2. Brainstorm as many ways as possible to select `dep_time`, `dep_delay`, `arr_time`, and `arr_delay` from `flights`.

`select()` and `relocate()` are different ways I've thus far learned to pick up or change the position of the columns. An example of using them would be:

```
flights |>
  select(dep_time, dep_delay, arr_time, arr_delay)
```

```
# A tibble: 336,776 x 4
  dep_time dep_delay arr_time arr_delay
  <int>     <dbl>   <int>     <dbl>
1     517         2     830         11
2     533         4     850         20
3     542         2     923         33
4     544        -1    1004        -18
5     554        -6     812        -25
6     554        -4     740         12
7     555        -5     913         19
8     557        -3     709        -14
9     557        -3     838         -8
10    558        -2     753          8
# i 336,766 more rows
```

```
flights|>
  relocate(dep_time, dep_delay, arr_time, arr_delay)
```

```
# A tibble: 336,776 x 19
  dep_time dep_delay arr_time arr_delay year month   day sched_dep_time
  <int>     <dbl>   <int>     <dbl> <int> <int> <int>          <int>
1     517         2     830         11  2013     1     1           515
2     533         4     850         20  2013     1     1           529
3     542         2     923         33  2013     1     1           540
4     544        -1    1004        -18  2013     1     1           545
5     554        -6     812        -25  2013     1     1           600
6     554        -4     740         12  2013     1     1           558
7     555        -5     913         19  2013     1     1           600
8     557        -3     709        -14  2013     1     1           600
9     557        -3     838         -8  2013     1     1           600
10    558        -2     753          8  2013     1     1           600
# i 336,766 more rows
# i 11 more variables: sched_arr_time <int>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

3. What happens. if you specify the name of the same variable multiple times in a `select()` call?

```
flights |>
  select(dep_time, arr_time, dep_time)
```

```
# A tibble: 336,776 x 2
  dep_time arr_time
  <int>      <int>
1     517        830
2     533        850
3     542        923
4     544       1004
5     554        812
6     554        740
7     555        913
8     557        709
9     557        838
10    558        753
# i 336,766 more rows
```

If you specify the same variable name multiple times in a select call, R just ignores the repeat and continues with the rest as usual.

4. What does the `any_of()` function do? Why might it be helpful in conjunction with this vector?

```
variables <- c("year", "month", "day", "dep_delay", "arr_delay")
```

The `any_of()` function is a selection helper which does not check for missing variables, but it does help with negative selections (making sure a variable is removed). This may be helpful for conjunction with the above vector, because it can remove or ignore the variables being asked for that aren't in the given data set.

5. Does the result of running the following code surprise you? How do the select helpers deal with upper and lower case by default? How can you change that default?

```
flights |> select(contains("TIME"))
```

```
# A tibble: 336,776 x 6
  dep_time sched_dep_time arr_time sched_arr_time air_time time_hour
  <int>      <int>      <int>      <int>      <dbl> <dtm>
1      517        515        830        819      227 2013-01-01 05:00:00
2      533        529        850        830      227 2013-01-01 05:00:00
3      542        540        923        850      160 2013-01-01 05:00:00
4      544        545       1004       1022      183 2013-01-01 05:00:00
5      554        600        812        837      116 2013-01-01 06:00:00
6      554        558        740        728      150 2013-01-01 05:00:00
7      555        600        913        854      158 2013-01-01 06:00:00
8      557        600        709        723       53 2013-01-01 06:00:00
9      557        600        838        846      140 2013-01-01 06:00:00
10     558        600        753        745      138 2013-01-01 06:00:00
# i 336,766 more rows
```

The result of running that code did not surprise me, it appears to have grabbed/selected the columns that contain the word “time” in them. The select helpers are case sensitive. To change this, you can use certain commands like `ignore_case=TRUE` which will ignore/match the characters to upper or lower case, whichever is neccessarey. `Ignore_case` came from section 15.5 of the textbook.

6. Rename `air_time` to `air_time_min` to indicate units of measurement and move it to the beginning of the data frame.

```
flights |>
  rename(air_time_min = air_time) |>
  relocate(air_time_min)
```

```
# A tibble: 336,776 x 19
  air_time_min year month   day dep_time sched_dep_time dep_delay arr_time
  <dbl> <int> <int> <int> <int>      <int>      <dbl>      <int>
1      227  2013     1     1      517        515         2      830
2      227  2013     1     1      533        529         4      850
3      160  2013     1     1      542        540         2      923
4      183  2013     1     1      544        545        -1     1004
5      116  2013     1     1      554        600        -6      812
6      150  2013     1     1      554        558        -4      740
7      158  2013     1     1      555        600        -5      913
8       53  2013     1     1      557        600        -3      709
9      140  2013     1     1      557        600        -3      838
10     138  2013     1     1      558        600        -2      753
# i 336,766 more rows
```

```
# i 11 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,  
#   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, distance <dbl>,  
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

7. Why doesn't the following work, and what does the error mean?

```
#flights |>  
  #select(tailnum) |>  
  #arrange(arr_delay)  
#> Error in `arrange()`:  
#>   In argument: `..1 = arr_delay`.  
#> Caused by error:  
#> ! object 'arr_delay' not found
```

This code doesn't work because `select` drops all of the other columns except `tailnum` so then when we go to `arrange(arr_delay)`, it no longer exists because we have already filtered out all of the columns except `tailnum` and there is no longer a variable to pick out called `arr_delay` in this code chunk.

I have commented out that last code chunk because R will not render my document with it.